Техническое задание на хакатон: прогноз продаж по складам и товарам

1. Цель задачи

Построить ML-модель для прогноза количества проданных единиц товара (`quantity`) по паре «склад/артикул» (`warehouse_name`, `article`) на горизонте 64 календарных дня. Прогноз должен быть ежедневным (1 строка = 1 день × 1 склад × 1 товар).

Задача ориентирована на практическое применение: модель должна учитывать сезонность, различия между категориями, влияние остатков на складе и типичные паттерны спроса в разрезе складов.

---

2. Исходные данные

Предоставляются два CSV-файла:

sales_dataset_hakaton.csv — продажи по дням

Поля:

- `date` — дата продажи (YYYY-MM-DD)
- `composite_division` — департамент
- `cities_name` — город
- `warehouse_name` — склад отгрузки
- `category_lvl_1`, `category_lvl_2`, `category_lvl_3` — категории товара (уровни 1–3)
- `article` — наименование товара (артикул)
- `characteristic` — характеристика товара
- `quantity` — количество проданных единиц
- `sales_sum` — сумма продаж
- `cost_price` — цена товара
- `discount_sum` — сумма скидки
- `volume_weight` — объёмный вес

stock_dataset_hakaton.csv — остатки на складах по дням

Поля:

- `date` — дата учёта остатков (YYYY-MM-DD)
- `warehouse_name` — наименование склада
- `char_article` — наименование товара (соответствует `article`)
- `stock` — остаток на складе в этот день

***Важно!** Данные анонимизированы и преобразованы для защиты коммерческой информации. Абсолютные значения продаж и цен не раскрывают реальные финансовые показатели компании.*

3. Целевая переменная и горизонт прогноза

- **Целевая переменная:** `quantity` (количество проданных единиц).
- **Горизонт прогноза:** 64 календарных дня вперёд от последней даты в обучающей выборке.
- **Разрез прогноза:** `warehouse_name` + `article`.
- **Агрегация департаментов:** все департаменты суммируются в рамках пары «склад — товар», то есть в прогнозе не нужно разделять по `composite_division`.

4. Формат результата (submission)

Участники загружают один CSV-файл с прогнозом на 64 дня.

Обязательные поля:

- `date` — дата прогноза (YYYY-MM-DD), ровно 64 уникальные даты подряд
- `warehouse_name` — наименование склада
- `article` — артикул товара
- `quantity` — прогнозное количество проданных единиц (без пропусков NaN)

Требования к файлу:

- Одна строка = один день × один склад × один товар.
- Должны быть прогнозы по всем комбинациям «склад × товар», которые встречались в обучающей выборке.
- Даты должны идти подряд без пропусков.
- Не допускаются пропуски (NaN) и отрицательные значения в `quantity`.
- 11 уникальных складов × 799 уникальных артикулов × 64 дня = **562 496 строк**.

Пример структуры:

```
date,warehouse_name,article,quantity
2025-10-01,warehouse_name_1,char_article_1,12.3
2025-10-01,warehouse_name_1,char_article_2,3.1
2025-10-01,warehouse_name_2,char_article_1,5.0
...
```

5. Метрики качества и скоринг

Оценка производится автоматически по **пяти метрикам** с весами, указанными ниже. Итоговый скор рассчитывается как взвешенная сумма нормализованных значений метрик. **Чем ниже скор — тем лучше.**

5.1. Метрики и веса

Метрика	Вес	Назначение		
SMAPE	**50%**	Главная метрика (относительная,		

		симметричная). Сравнивает ошибки по разным товарам и складам.		
WAPE	**20%**	Устойчив к нулевым значениям, не «взрывается» на малых продажах. Страхует SMAPE на редких товарах.		
RMSE	**15%**	Штрафует крупные выбросы. Важен для ходовых товаров, где абсолютная ошибка бьёт по бизнесу.		
**\	Bias\	**	**10%**	Ловит систематический перекос (модель всегда завышает или занижает). Античит против занижения прогноза.
RMSLE	**5%**	Чувствителен к ошибкам на малых значениях, сглаживает выбросы. Полезен для редких артикулов.		

5.2. Формулы метрик

SMAPE (Symmetric Mean Absolute Percentage Error):

$$ \text{SMAPE} = \frac{1}{N} \sum_{i=1}^{N} \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|) / 2} \times 100 $$

- Если $y_i = 0$ и $\hat{y}_i = 0$: ошибка = 0 (идеальный прогноз).
- Если $y_i = 0$ и $\hat{y}_i > 0$: ошибка = 200% (максимальная).
- Если $y_i > 0$ и $\hat{y}_i = 0$: ошибка = 200%.

- Считается **по каждой серии** (склад × товар) и усредняется — каждая серия имеет равный вес.

WAPE (Weighted Absolute Percentage Error):

$$ \text{WAPE} = \frac{\sum_{i=1}^{N} |y_i - \hat{y}_i|}{\sum_{i=1}^{N} |y_i|} \times 100 $$

- Считается **глобально** по всем точкам — ходовые товары получают естественный больший вес.

RMSE (Root Mean Squared Error):

$$ \text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2} $$

- Считается **глобально** по всем точкам.

Bias (Mean Percentage Error, по модулю):

$$ \text{Bias} = \left| \frac{\bar{\hat{y}} - \bar{y}}{\bar{y}} \right| \times 100 $$

- Если $\bar{y} = 0$ для серии — серия исключается из расчёта Bias.
- Считается **по каждой серии** и усредняется.

RMSLE (Root Mean Squared Logarithmic Error):

$$ \text{RMSLE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} \left(\ln(\hat{y}_i + 1) - \ln(y_i + 1) \right)^2} $$

- Используется `log1p` для безопасности при нулевых значениях.
- Считается **глобально**.

5.3. Нормализация

Метрики находятся в разных шкалах, поэтому перед взвешиванием они нормализуются. Поддерживаются два режима:

Min-max нормализация (по умолчанию):

- За 0 принимается значение метрики у «идеального» прогноза (0 для всех метрик).
- За 1 — значение у наивного бейзлайна (прогноз последним известным значением).
- Всё, что хуже бейзлайна, клиппится на 1.
- Требует наличия baseline-файла.

Ранговая нормализация:

- Считается метрика для каждого участника.
- Присваивается ранг (1 — лучший, N — худший).
- Нормируется в $[0; 1]$: $r_{\text{norm}} = \frac{\text{rank} - 1}{N - 1}$.
- Не требует бейзлайна, устойчива к выбросам.

5.4. Итоговая формула скоринга

$$ \text{Score} = 0{,}50 \cdot \text{SMAPE}_{\text{norm}} + 0{,}20 \cdot \text{WAPE}_{\text{norm}} + 0{,}15 \cdot \text{RMSE}_{\text{norm}} + 0{,}10 \cdot |\text{Bias}|_{\text{norm}} + 0{,}05 \cdot \text{RMSLE}_{\text{norm}} $$

Чем **ниже** скор — тем лучше.

5.5. Обработка краевых случаев

- **Факт = 0 и прогноз = 0:** SMAPE = 0 (идеальный прогноз).
- **Факт = 0, прогноз > 0:** SMAPE = 200%.
- **Факт > 0, прогноз = 0:** SMAPE = 200%.
- **Для Bias:** если $\bar{y} = 0$ (все фактические значения нулевые для серии), серия исключается из расчёта Bias.
- **Для RMSLE:** используется `log1p` (натуральный логарифм от $x + 1$) — безопасно для нулей.
- **Пропуски в submission:** заполняются нулями с предупреждением.

---

6. Особенности подготовки данных и ограничения

- **Анонимизация и преобразования.** Данные уже прошли анонимизацию и нелинейные преобразования (масштабирование, смещение, нелинейность). Не пытайтесь восстановить реальные абсолютные значения. Модель должна обучаться на предоставленном представлении данных.
- **Нулевые и редкие значения.** В данных возможны нулевые продажи и товары с редкими продажами. Модель должна корректно работать и с такими случаями.
- **Остатки на складе.** Поле `stock` из второго датасета может быть полезно для учёта эффекта «нет товара — нет продаж».
- **Категориальные признаки.** Разрешается использовать `category_lvl_1/2/3`, `warehouse_name`, `cities_name` и т. п. для построения признаков.
- **Временные признаки.** Рекомендуется извлекать день недели, месяц, признаки сезонности, праздники (если есть внешний календарь) и т. д.

---

7. Рекомендации по подходу

Участники могут использовать любые модели и подходы:

- классические временные ряды (ETS, ARIMA, Prophet) по отдельным сериям;
- градиентный бустинг (CatBoost, XGBoost, LightGBM) с признаками по времени;
- нейросетевые модели (если укладываются в лимиты по времени/памяти);
- ансамбли и стекинг.

Практические советы:

- Стройте прогноз по разрезу «склад × товар» (это минимальная единица).
- Учитывайте, что у разных товаров и складов разные паттерны: одни товары продаются каждый день, другие — эпизодически.
- Используйте остатки (`stock`) как признак: низкий остаток может коррелировать с упущенными продажами или с изменением динамики.

- При необходимости применяйте логарифмирование/иные трансформации таргета внутри своего пайплайна — это не запрещено.

---

8. Ограничения и правила участия

- Технические ограничения: ОЗУ 64 GB, использование GPU по желанию.
- Запрещено использовать любые внешние данные из закрытых источников, не предоставленные организаторами.
- Запрещено пытаться «восстановить» исходные абсолютные значения и строить логику решения на предположении о реальных суммах.
- Код и описание решения (notebook/скрипт) должны быть предоставлены в архиве вместе с прогнозом (формат и требования к коду будут объявлены отдельно).
- Прогноз должен быть детерминированным: при повторном запуске на тех же данных результат не должен меняться.

---

9. Скрипт автоматического скоринга

Ниже приведён шаблонный скрипт `score.py` для автоматического подсчёта метрик и формирования лидерборда. Скрипт поддерживает как одиночные, так и множественные сабмиссии, два режима нормализации (min-max и ранговую), валидацию формата и обработку краевых случаев. Ожидается применение именно ранговой версии.

9.1. Использование

```
# Одиночный сабмит (с бейзлайном для нормализации)
python score.py --truth ground_truth.csv --submissions my_forecast.csv --baseline
naive_forecast.csv

# Сравнение нескольких моделей
python score.py --truth ground_truth.csv --submissions model_a.csv model_b.csv model_c.csv -
-baseline naive_forecast.csv

# С ранговой нормализацией (без бейзлайна)
python score.py --truth ground_truth.csv --submissions model_a.csv model_b.csv --method rank

# Сохранить результаты в CSV
python score.py --truth ground_truth.csv --submissions model_a.csv model_b.csv --output
leaderboard.csv
```

9.2. Пример полного кода скрипта

```
#!/usr/bin/env python3
"""
Автоматический скоринг для хакатона «Прогноз продаж по складам и товарам».

Метрики и веса:
    SMAPE  — 50%   (считается по сериям, усредняется)
    WAPE   — 20%   (глобально)
    RMSE   — 15%   (глобально)
    Bias   — 10%   (|MPE| по сериям, усредняется)
```

```
    RMSLE  — 5%    (глобально, log1p)

Чем ниже итоговый скор — тем лучше.

Режимы нормализации:
    minmax — 0 = идеальный прогноз, 1 = наивный бейзлайн
    rank   — ранги участников от 0 (лучший) до 1 (худший)

Usage:
    python score.py --truth ground_truth.csv \
        --submissions model_a.csv model_b.csv \
        --baseline naive_forecast.csv \
        --output leaderboard.csv
"""

import argparse
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from typing import Dict, List, Optional

# ─────────────────────────────────────────────────────────────
# Константы
# ─────────────────────────────────────────────────────────────

METRIC_WEIGHTS: Dict[str, float] = {
    "smape": 0.50,
    "wape":  0.20,
    "rmse":  0.15,
    "bias":  0.10,
    "rmsle": 0.05,
}

REQUIRED_COLUMNS = ["date", "warehouse_name", "article", "quantity"]
NUM_WAREHOUSES = 11
NUM_ARTICLES = 799
FORECAST_HORIZON = 64
EXPECTED_ROWS = NUM_WAREHOUSES * NUM_ARTICLES * FORECAST_HORIZON  # 562 496


# ─────────────────────────────────────────────────────────────
# Метрики
# ─────────────────────────────────────────────────────────────

def smape_series(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """SMAPE для одной серии. Возвращает значение в процентах (0..200)."""
    numerator = np.abs(y_true - y_pred)
    denominator = (np.abs(y_true) + np.abs(y_pred)) / 2.0
    mask = denominator > 0
    if not mask.any():
        return 0.0
    return float(np.mean(numerator[mask] / denominator[mask]) * 100.0)
```

```
def smape(y_true: np.ndarray, y_pred: np.ndarray,
          series_ids: np.ndarray) -> float:
    """SMAPE по сериям (склад × товар), усреднённый."""
    unique = np.unique(series_ids)
    values = []
    for sid in unique:
        mask = series_ids == sid
        values.append(smape_series(y_true[mask], y_pred[mask]))
    return float(np.mean(values))


def wape(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """WAPE — глобально. Возвращает значение в процентах."""
    denom = np.sum(np.abs(y_true))
    if denom == 0:
        return 0.0
    return float(np.sum(np.abs(y_true - y_pred)) / denom * 100.0)


def rmse(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """RMSE — глобально."""
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))


def bias_abs_series(y_true: np.ndarray, y_pred: np.ndarray) -> Optional[float]:
    """|Bias| для одной серии. None если mean(y_true) == 0."""
    mean_y = np.mean(y_true)
    if mean_y == 0:
        return None
    return float(abs(np.mean(y_pred) - mean_y) / mean_y * 100.0)


def bias_abs(y_true: np.ndarray, y_pred: np.ndarray,
             series_ids: np.ndarray) -> float:
    """|Bias| по сериям, усреднённый. Серии с mean(y)=0 исключаются."""
    unique = np.unique(series_ids)
    values = []
    for sid in unique:
        mask = series_ids == sid
        val = bias_abs_series(y_true[mask], y_pred[mask])
        if val is not None:
            values.append(val)
    if not values:
        return 0.0
    return float(np.mean(values))


def rmsle(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """RMSLE — глобально, с log1p для безопасности."""
    return float(np.sqrt(np.mean((np.log1p(y_pred) - np.log1p(y_true)) ** 2)))


def compute_all_metrics(
    df_true: pd.DataFrame,
```

```
    df_pred: pd.DataFrame
) -> Dict[str, float]:
    """Считает все 5 метрик для пары (truth, prediction)."""
    df = df_true.merge(
        df_pred, on=["date", "warehouse_name", "article"], how="left",
        suffixes=("_true", "_pred")
    )
    nan_count = df["quantity_pred"].isna().sum()
    if nan_count > 0:
        print(f"  ⚠  Пропуски в submission: {nan_count} строк. "
              f"Заполнены нулями.")
    df["quantity_pred"] = df["quantity_pred"].fillna(0).clip(lower=0)

    y_true = df["quantity_true"].values.astype(float)
    y_pred = df["quantity_pred"].values.astype(float)
    series_ids = (
        df["warehouse_name"].astype(str) + "|" + df["article"].astype(str)
    ).values

    return {
        "smape": smape(y_true, y_pred, series_ids),
        "wape":  wape(y_true, y_pred),
        "rmse":  rmse(y_true, y_pred),
        "bias":  bias_abs(y_true, y_pred, series_ids),
        "rmsle": rmsle(y_true, y_pred),
    }


# ──────────────────────────────────────────────────────────────
# Нормализация
# ──────────────────────────────────────────────────────────────

def normalize_minmax(
    metrics: Dict[str, Dict[str, float]],
    baseline_metrics: Dict[str, float]
) -> Dict[str, Dict[str, float]]:
    """Min-max: 0 = идеал, 1 = бейзлайн. Всё хуже бейзлайна → 1."""
    normalized = {}
    for name, m in metrics.items():
        normalized[name] = {}
        for key in METRIC_WEIGHTS:
            val = m[key]
            base = baseline_metrics[key]
            if base == 0:
                norm = 0.0 if val == 0 else 1.0
            else:
                norm = min(val / base, 1.0)
            normalized[name][key] = norm
    return normalized


def normalize_rank(
    metrics: Dict[str, Dict[str, float]]
) -> Dict[str, Dict[str, float]]:
    """Ранговая нормализация: 0 = лучший, 1 = худший."""
```

```
    names = list(metrics.keys())
    n = len(names)
    normalized = {name: {} for name in names}
    for key in METRIC_WEIGHTS:
        values = [metrics[name][key] for name in names]
        order = np.argsort(values)
        ranks = np.empty(n, dtype=float)
        ranks[order] = np.arange(n, dtype=float)
        for i, name in enumerate(names):
            normalized[name][key] = (
                ranks[i] / (n - 1) if n > 1 else 0.0
            )
    return normalized


# ──────────────────────────────────────────────────────────────
# Валидация формата submission
# ──────────────────────────────────────────────────────────────

def validate_submission(df: pd.DataFrame, name: str = "submission") -> List[str]:
    """Проверяет формат submission и возвращает список предупреждений."""
    warnings_list = []

    missing_cols = set(REQUIRED_COLUMNS) - set(df.columns)
    if missing_cols:
        warnings_list.append(f"Отсутствуют колонки: {missing_cols}")
        return warnings_list

    nan_count = df["quantity"].isna().sum()
    if nan_count > 0:
        warnings_list.append(f"NaN в quantity: {nan_count} строк")

    neg_count = (df["quantity"] < 0).sum()
    if neg_count > 0:
        warnings_list.append(
            f"Отрицательные значения в quantity: {neg_count} строк"
        )

    if len(df) != EXPECTED_ROWS:
        warnings_list.append(
            f"Ожидается {EXPECTED_ROWS} строк, получено {len(df)}"
        )

    dates = pd.to_datetime(df["date"], errors="coerce")
    unique_dates = dates.dropna().unique()
    if len(unique_dates) != FORECAST_HORIZON:
        warnings_list.append(
            f"Ожидается {FORECAST_HORIZON} уникальных дат, "
            f"получено {len(unique_dates)}"
        )

    if len(unique_dates) == FORECAST_HORIZON:
        sorted_dates = pd.to_datetime(np.sort(unique_dates))
        gaps = sorted_dates.to_series().diff().dropna()
        if (gaps != pd.Timedelta(days=1)).any():
```

```
            warnings_list.append("Даты не идут подряд")

    combos = df.groupby(["warehouse_name", "article"]).ngroups
    expected_combos = NUM_WAREHOUSES * NUM_ARTICLES
    if combos != expected_combos:
        warnings_list.append(
            f"Ожидается {expected_combos} уникальных комбинаций "
            f"склад × товар, получено {combos}"
        )

    return warnings_list


# ──────────────────────────────────────────────────────────────
# Главная функция
# ──────────────────────────────────────────────────────────────

def main():
    parser = argparse.ArgumentParser(
        description="Скоринг для хакатона «Прогноз продаж»"
    )
    parser.add_argument(
        "--truth", required=True,
        help="Путь к ground truth CSV (date, warehouse_name, article, quantity)"
    )
    parser.add_argument(
        "--submissions", nargs="+", required=True,
        help="Пути к submission CSV (один или несколько)"
    )
    parser.add_argument(
        "--baseline",
        help="Путь к baseline CSV для min-max нормализации"
    )
    parser.add_argument(
        "--method", choices=["minmax", "rank"], default="minmax",
        help="Метод нормализации (по умолчанию: minmax)"
    )
    parser.add_argument(
        "--output", help="Путь для сохранения лидерборда в CSV"
    )
    parser.add_argument(
        "--weights", type=str, default=None,
        help=(
            "Веса метрик в формате JSON: "
            '{"smape":0.5,"wape":0.2,"rmse":0.15,"bias":0.1,"rmsle":0.05}'
        )
    )

    args = parser.parse_args()

    if args.weights:
        import json
        custom_weights = json.loads(args.weights)
        global METRIC_WEIGHTS
        METRIC_WEIGHTS = custom_weights
```

```
    print(f"Веса метрик переопределены: {METRIC_WEIGHTS}")

print("=" * 70)
print("СКООРИНГ ХАКАТОНА «ПРОГНОЗ ПРОДАЖ»")
print("=" * 70)
print(f"Веса метрик: {METRIC_WEIGHTS}")
print(f"Метод нормализации: {args.method}")
print(f"Ground truth: {args.truth}")
print(f"Submissions: {args.submissions}")
print()

df_truth = pd.read_csv(args.truth)
df_truth["date"] = pd.to_datetime(df_truth["date"])
print(f"Ground truth загружен: {len(df_truth)} строк, "
      f"{df_truth['date'].nunique()} дат")

baseline_metrics = None
if args.method == "minmax":
    if args.baseline is None:
        print("⚠  Метод minmax требует --baseline. "
              "Переключаюсь на rank.")
        args.method = "rank"
    else:
        df_base = pd.read_csv(args.baseline)
        df_base["date"] = pd.to_datetime(df_base["date"])
        print(f"Baseline загружен: {len(df_base)} строк")
        baseline_metrics = compute_all_metrics(df_truth, df_base)
        print(f"Метрики baseline: {baseline_metrics}")

all_metrics: Dict[str, Dict[str, float]] = {}
for sub_path in args.submissions:
    name = Path(sub_path).stem
    print(f"\n--- {name} ---")
    df_sub = pd.read_csv(sub_path)
    df_sub["date"] = pd.to_datetime(df_sub["date"])

    warns = validate_submission(df_sub, name)
    for w in warns:
        print(f"  ⚠  {w}")
    if any("Отсутствуют колонки" in w for w in warns):
        print(f"  ✗  Пропуск {name} — нет обязательных колонок")
        continue

    m = compute_all_metrics(df_truth, df_sub)
    all_metrics[name] = m
    print(f"  SMAPE = {m['smape']:.4f}%")
    print(f"  WAPE  = {m['wape']:.4f}%")
    print(f"  RMSE  = {m['rmse']:.4f}")
    print(f"  |Bias|= {m['bias']:.4f}%")
    print(f"  RMSLE = {m['rmsle']:.4f}")

if not all_metrics:
    print("\n✗  Нет валидных submissions для оценки.")
    sys.exit(1)
```

```
    if args.method == "minmax" and baseline_metrics is not None:
        normalized = normalize_minmax(all_metrics, baseline_metrics)
    else:
        normalized = normalize_rank(all_metrics)

    print("\n" + "=" * 70)
    print("ЛИДЕРБОРД")
    print("=" * 70)
    leaderboard = []
    for name in all_metrics:
        score = sum(
            METRIC_WEIGHTS[k] * normalized[name][k]
            for k in METRIC_WEIGHTS
        )
        row = {"submission": name, "score": score}
        for k in METRIC_WEIGHTS:
            row[f"{k}_raw"] = all_metrics[name][k]
            row[f"{k}_norm"] = normalized[name][k]
        leaderboard.append(row)

    df_lb = pd.DataFrame(leaderboard).sort_values("score")
    print(df_lb.to_string(index=False, float_format="%.4f"))

    if args.output:
        df_lb.to_csv(args.output, index=False)
        print(f"\nЛидерборд сохранён в {args.output}")

    print(f"\nПобедитель: {df_lb.iloc[0]['submission']} "
          f"(score = {df_lb.iloc[0]['score']:.4f})")


if __name__ == "__main__":
    main()
```

---

10. Задание со звёздочкой: контейнер с моделью и API для прогноза

Это дополнительное задание для команд, желающих продемонстрировать умение упаковывать ML-модели в production-ready сервис. Выполняется **по желанию**, оценивается отдельно (до 8 баллов).

10.1. Постановка

Участники должны предоставить **Git-репозиторий** с Docker-контейнером, внутри которого развёрнута итоговая модель прогноза. Контейнер должен предоставлять HTTP-API на базе FastAPI (или аналогичного фреймворка) с тремя эндпоинтами для получения прогнозов.

На вход модели подаются два CSV-файла с данными за последние полгода (≈180 дней):

1. `sales` — продажи по дням в формате `sales_dataset_hakaton.csv` (те же колонки, что и в обучающей выборке).

2. `stock` — остатки на складах по дням в формате `stock_dataset_hakaton.csv` (те же колонки, что и в обучающей выборке).

Модель должна прочитать эти файлы, рассчитать прогноз на 64 дня вперёд от последней даты в переданных данных и вернуть результат. Контейнер не имеет доступа к обучающей выборке — все данные для построения прогноза приходят через API.

10.2. Технические ограничения

Параметр	Ограничение
Время сборки образа	до 60 минут
Объём RAM	до 64 GB
GPU	доступен (CUDA)
Размер образа	до 30 GB
Время инференса (POST)	до 60 минут на полный набор (562 496 строк)
Время инференса (GET)	до 10 минут на один запрос
Сетевой доступ при инференсе	**запрещён** (модель должна быть полностью автономной)
Детерминированность	при повторном запросе с теми же данными результат не должен меняться

10.3. Эндпоинты API

POST `/predict` — полный прогноз по загруженным данным

Запрос:

- Content-Type: `multipart/form-data`
- Два файла:
- `sales` — CSV в формате `sales_dataset_hakaton.csv` (продажи за последние ~180 дней / полгода)
- `stock` — CSV в формате `stock_dataset_hakaton.csv` (остатки за последние ~180 дней / полгода)

Ожидания по входным данным:

- Глубина истории в переданных файлах — **не менее 180 дней** (иначе 422). Переданных данных может быть больше полугода — модель рассчитывает прогноз по всем предоставленным данным.
- Колонки должны совпадать с обучающей выборкой (см. раздел 2).
- Последняя дата в `sales` определяет стартовую точку прогноза: 64 календарных дня вперёд.

Ответ:

- Content-Type: `text/csv`
- Тело: CSV с прогнозом на 64 дня в формате submission:

```
date,warehouse_name,article,quantity
2025-10-01,warehouse_name_1,char_article_1,12.3
...
```

Пример curl:

```
curl -X POST http://localhost:8000/predict \
  -F "sales=@sales_last_6mo.csv" \
  -F "stock=@stock_last_6mo.csv" \
  -o forecast.csv
```

GET /predict — прогноз по конкретному артикулу и/или складу

Query-параметры:

Параметр	Тип	Обязательный	Описание
article	string	нет	Артикул товара
warehouse	string	нет	Наименование склада

- Хотя бы один параметр обязателен. Если не указан ни один — возвращается 400 Bad Request.
- Если указан только article — прогноз по всем складам для этого артикула.
- Если указан только warehouse — прогноз по всем артикулам для этого склада.
- Если указаны оба — прогноз по конкретной паре «склад × товар».

Ответ:

- Content-Type: text/csv
- Тело: CSV с прогнозом в формате submission (те же колонки, что и в основном файле):

```
date,warehouse_name,article,quantity
2025-10-01,warehouse_name_1,char_article_1,12.3
2025-10-02,warehouse_name_1,char_article_1,11.8
2025-10-03,warehouse_name_1,char_article_1,10.5
...
```

Примеры curl:

```
# Прогноз по конкретному артикулу на всех складах
curl "http://localhost:8000/predict?article=char_article_1" -o article_forecast.csv

# Прогноз по конкретному складу для всех товаров
curl "http://localhost:8000/predict?warehouse=warehouse_name_3" -o warehouse_forecast.csv

# Прогноз по конкретной паре склад × товар
curl "http://localhost:8000/predict?article=char_article_1&warehouse=warehouse_name_3" -o
pair_forecast.csv
```

GET /predict/all — весь прогноз целиком

Query-параметры:

Параметр	Тип	По умолчанию	Описание
format	string	csv	Формат ответа: csv

			или `json`

- Возвращает полный прогноз по всем комбинациям «склад × товар» на 64 дня (562 496 строк).
- Формат `csv` — потоковая отдача файла (Content-Type: `text/csv`).
- Формат `json` — JSON-массив объектов (может быть тяжёлым, ~100+ MB; рекомендуется `csv`).

Примеры curl:

```
# Скачать весь прогноз в CSV
curl "http://localhost:8000/predict/all?format=csv" -o full_forecast.csv

# Получить весь прогноз в JSON
curl "http://localhost:8000/predict/all?format=json" | jq . > full_forecast.json
```

10.4. Валидация входных данных

При получении данных через POST `/predict` сервер должен валидировать:

3. **Наличие обязательных колонок** в обоих CSV (см. раздел 2).
4. **Формат дат** — YYYY-MM-DD, парсинг без ошибок.
5. **Минимальная глубина истории** — не менее 180 дней (иначе `422 Unprocessable Entity` с сообщением).
6. **Отрицательные значения** — если `quantity` или `stock` содержат отрицательные числа, вернуть 422 с описанием проблемы.
7. **Пустые файлы** — вернуть `400 Bad Request`.

Все ошибки возвращаются в формате:

```
{
  "detail": "Описание ошибки",
  "code": "VALIDATION_ERROR"
}
```

10.5. Ожидаемая структура репозитория

```
forecast-container/
├── Dockerfile
├── docker-compose.yml         # (опционально, если нужны GPU-томы)
├── requirements.txt
├── README.md
├── app/
│   ├── main.py               # FastAPI-приложение
│   ├── predictor.py          # Логика загрузки модели и инференса
│   └── validators.py         # Валидация входных CSV
├── model/
│   ├── model.pkl             # (или .pt, .cbm, .lgb — формат на выбор)
│   ├── preprocessor.pkl      # Пайплайн предобработки
│   └── config.json           # Конфиг модели (признаки, параметры)
├── data/
│   ├── sample_sales.csv      # Пример входных данных (первые 100 строк)
│   └── sample_stock.csv      # Пример входных данных (первые 100 строк)
└── tests/
    └── test_api.sh           # Скрипт самопроверки эндпоинтов
```

10.6. Требования к Dockerfile

```
FROM python:3.11-slim

# Установка системных зависимостей
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Установка Python-зависимостей
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Копирование приложения и модели
COPY app/ ./app/
COPY model/ ./model/

# Модель загружается в память при старте контейнера
EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

10.7. Шаблон FastAPI-приложения (app/main.py)

```
import io
from pathlib import Path
from typing import Optional

import pandas as pd
from fastapi import FastAPI, File, HTTPException, Query
from fastapi.responses import StreamingResponse, JSONResponse

from .predictor import Predictor
from .validators import validate_sales_csv, validate_stock_csv

app = FastAPI(
    title="Hackathon Forecast API",
    description="Прогноз продаж по складам и товарам",
    version="1.0.0",
)

# Загрузка модели при старте
predictor = Predictor.load(Path("model"))


@app.get("/health")
async def health():
    return {"status": "ok"}


@app.post("/predict")
async def predict_post(
    sales: bytes = File(...),
    stock: bytes = File(...),
```

```
):
    """
    Принимает два CSV (sales + stock) и возвращает CSV с прогнозом на 64 дня.
    """
    try:
        df_sales = pd.read_csv(io.BytesIO(sales))
        df_stock = pd.read_csv(io.BytesIO(stock))
    except Exception as e:
        raise HTTPException(
            status_code=400,
            detail=f"Ошибка парсинга CSV: {str(e)}"
        )

    errors = validate_sales_csv(df_sales) + validate_stock_csv(df_stock)
    if errors:
        raise HTTPException(
            status_code=422,
            detail={"errors": errors, "code": "VALIDATION_ERROR"}
        )

    df_forecast = predictor.predict(df_sales, df_stock)

    output = io.StringIO()
    df_forecast.to_csv(output, index=False)
    output.seek(0)

    return StreamingResponse(
        iter([output.getvalue()]),
        media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=forecast.csv"}
    )


@app.get("/predict")
async def predict_get(
    article: Optional[str] = Query(None, description="Артикул товара"),
    warehouse: Optional[str] = Query(None, description="Наименование склада"),
):
    """
    Возвращает прогноз по конкретному артикулу и/или складу в формате CSV.
    Хотя бы один параметр обязателен.
    """
    if not article and not warehouse:
        raise HTTPException(
            status_code=400,
            detail="Укажите хотя бы один параметр: article или warehouse"
        )

    df_forecast = predictor.get_cached_forecast()

    mask = pd.Series(True, index=df_forecast.index)
    if article:
        mask &= df_forecast["article"] == article
    if warehouse:
        mask &= df_forecast["warehouse_name"] == warehouse
```

```
    result = df_forecast[mask].copy()

    output = io.StringIO()
    result.to_csv(output, index=False)
    output.seek(0)

    return StreamingResponse(
        iter([output.getvalue()]),
        media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=filtered_forecast.csv"}
    )


@app.get("/predict/all")
async def predict_all(
    format: str = Query("csv", description="Формат ответа: csv или json"),
):
    """
    Возвращает весь прогноз целиком (562 496 строк).
    """
    df_forecast = predictor.get_cached_forecast()

    if format == "csv":
        output = io.StringIO()
        df_forecast.to_csv(output, index=False)
        output.seek(0)
        return StreamingResponse(
            iter([output.getvalue()]),
            media_type="text/csv",
            headers={
                "Content-Disposition":
                    "attachment; filename=full_forecast.csv"
            }
        )
    elif format == "json":
        result = df_forecast.to_dict(orient="records")
        return JSONResponse(content=result)
    else:
        raise HTTPException(
            status_code=400,
            detail="Параметр format должен быть 'csv' или 'json'"
        )
```

10.8. Шаблон валидатора (app/validators.py)

```
import pandas as pd
from typing import List

SALES_REQUIRED_COLS = [
    "date", "warehouse_name", "article", "quantity",
    "sales_sum", "cost_price", "discount_sum", "volume_weight",
    "composite_division", "cities_name",
    "category_lvl_1", "category_lvl_2", "category_lvl_3",
    "characteristic",
]
```

```
STOCK_REQUIRED_COLS = ["date", "warehouse_name", "char_article", "stock"]

MIN_HISTORY_DAYS = 180


def validate_sales_csv(df: pd.DataFrame) -> List[str]:
    errors = []
    missing = set(SALES_REQUIRED_COLS) - set(df.columns)
    if missing:
        errors.append(f"Отсутствуют колонки в sales: {missing}")
        return errors

    try:
        dates = pd.to_datetime(df["date"], format="%Y-%m-%d")
    except Exception:
        errors.append("Неверный формат даты в sales (ожидается YYYY-MM-DD)")
        return errors

    history_days = (dates.max() - dates.min()).days
    if history_days < MIN_HISTORY_DAYS:
        errors.append(
            f"Недостаточно истории: {history_days} дней, "
            f"минимум {MIN_HISTORY_DAYS}"
        )

    if (df["quantity"] < 0).any():
        errors.append("Найдены отрицательные значения в quantity")

    return errors


def validate_stock_csv(df: pd.DataFrame) -> List[str]:
    errors = []
    missing = set(STOCK_REQUIRED_COLS) - set(df.columns)
    if missing:
        errors.append(f"Отсутствуют колонки в stock: {missing}")
        return errors

    try:
        pd.to_datetime(df["date"], format="%Y-%m-%d")
    except Exception:
        errors.append("Неверный формат даты в stock (ожидается YYYY-MM-DD)")
        return errors

    if (df["stock"] < 0).any():
        errors.append("Найдены отрицательные значения в stock")

    return errors
```

10.9. Шаблон предиктора (app/predictor.py)

```
import json
from pathlib import Path
from typing import Optional
import pandas as pd
```

```
class Predictor:
    """Обёртка над моделью для инференса."""

    def __init__(self, model, preprocessor, config):
        self.model = model
        self.preprocessor = preprocessor
        self.config = config
        self._cached_forecast: Optional[pd.DataFrame] = None

    @classmethod
    def load(cls, model_dir: Path) -> "Predictor":
        """Загрузка модели из директории. Реализовать под свой формат."""
        config = json.loads((model_dir / "config.json").read_text())

        # Пример для pickled-модели:
        import pickle
        model = pickle.loads((model_dir / "model.pkl").read_bytes())
        preprocessor = pickle.loads(
            (model_dir / "preprocessor.pkl").read_bytes()
        )
        return cls(model, preprocessor, config)

    def predict(
        self, df_sales: pd.DataFrame, df_stock: pd.DataFrame
    ) -> pd.DataFrame:
        """
        Основная логика прогноза.
        Принимает sales и stock, возвращает DataFrame в формате submission.
        """
        # ── Заглушка: замените на свою логику ──
        # 1. Объединить sales + stock
        # 2. Сгенерировать признаки
        # 3. Применить модель
        # 4. Собрать submission-формат

        forecast_dates = pd.date_range(
            start=df_sales["date"].max(),
            periods=65,
            freq="D"
        )[1:]  # 64 дня

        combos = (
            df_sales[["warehouse_name", "article"]]
            .drop_duplicates()
        )

        rows = []
        for _, row in combos.iterrows():
            for d in forecast_dates:
                rows.append({
                    "date": d.strftime("%Y-%m-%d"),
                    "warehouse_name": row["warehouse_name"],
                    "article": row["article"],
```

```
                    "quantity": 0.0,  # ← замените на прогноз модели
                })

        df_forecast = pd.DataFrame(rows)
        self._cached_forecast = df_forecast
        return df_forecast

    def get_cached_forecast(self) -> pd.DataFrame:
        """Возвращает кэшированный прогноз (для GET-эндпоинтов)."""
        if self._cached_forecast is None:
            raise RuntimeError(
                "Прогноз не рассчитан. Сначала вызовите POST /predict."
            )
        return self._cached_forecast
```

10.10. Шаблон скрипта самопроверки (`tests/test_api.sh`)

```
#!/bin/bash
# Самопроверка API. Запустите после поднятия контейнера.
# Usage: bash tests/test_api.sh [BASE_URL]

set -e
BASE_URL="${1:-http://localhost:8000}"

echo "=== 1. Health check ==="
curl -sf "$BASE_URL/health" | jq .
echo "OK"
echo

echo "=== 2. POST /predict ==="
curl -sf -X POST "$BASE_URL/predict" \
  -F "sales=@data/sample_sales.csv" \
  -F "stock=@data/sample_stock.csv" \
  -o /tmp/forecast.csv
echo "Получен файл: $(wc -l < /tmp/forecast.csv) строк"
head -3 /tmp/forecast.csv
echo "OK"
echo

echo "=== 3. GET /predict?article= ==="
curl -sf "$BASE_URL/predict?article=char_article_1" | head -3
echo "OK"
echo

echo "=== 4. GET /predict?warehouse= ==="
curl -sf "$BASE_URL/predict?warehouse=warehouse_name_1" | head -3
echo "OK"
echo

echo "=== 5. GET /predict?article=&warehouse= ==="
curl -sf "$BASE_URL/predict?article=char_article_1&warehouse=warehouse_name_1" | head -3
echo "OK"
echo

echo "=== 6. GET /predict/all?format=csv ==="
curl -sf "$BASE_URL/predict/all?format=csv" -o /tmp/full_forecast.csv
```

```
echo "Получен файл: $(wc -l < /tmp/full_forecast.csv) строк"
echo "OK"
echo

echo "=== 7. GET /predict/all?format=json ==="
curl -sf "$BASE_URL/predict/all?format=json" | jq '.[0:3]'
echo "OK"
echo

echo "=== 8. Error: GET /predict без параметров ==="
curl -sf "$BASE_URL/predict" 2>&1 || true
echo "Ожидается 400"
echo

echo "=== Все тесты пройдены! ==="
```

10.11. Шкала оценки

Критерий	**Баллы**
Контейнер собирается без ошибок, API поднимается	1
POST `/predict` корректно принимает CSV и возвращает прогноз нужного формата	2
GET `/predict` с query-параметрами `article` и/или `warehouse` работает корректно	1
GET `/predict/all` отдаёт весь прогноз целиком (CSV и/или JSON)	1
Качество прогноза из контейнера не хуже, чем в основном submission	2
Время инференса в пределах лимита (60 мин для POST, 10 мин для GET)	0,5
Документация в README, чистота кода, корректный `.dockerignore`	0,5
Итого	**8**

10.12. README репозитория — обязательные разделы

8. **Описание решения** — кратко: какая модель, какие признаки, какой подход.
9. **Инструкция по сборке** — `docker build` команда, требования к Docker (версия, GPU-поддержка).
10. **Инструкция по запуску** — `docker run` с указанием портов и GPU-флагов.
11. **Описание API** — список эндпоинтов с примерами curl.
12. **Структура репозитория** — дерево директорий с комментариями.
13. **Известные ограничения** — что не работает, что требует доработки.